

โปรเจค: CP353002 Principles of Software Design and Development (Spring Boot)

1. วัตถุประสงค์การเรียนรู้

- ออกแบบและพัฒนาระบบด้วย Spring Boot ตามสถาปัตยกรรมแบบ Layered Architecture
- ประยุกต์ใช้ SOLID Principles และ Design Patterns ได้อย่างเหมาะสมและอธิบายเหตุผลได้
- ออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีความสัมพันธ์ One-to-One และ One-to-Many
- พัฒนา REST API และ Frontend ที่เชื่อมต่อกันได้จริง
- ทำงานเป็นทีมด้วย Git/GitHub อย่างมีวินัย และ Deploy ระบบขึ้นใช้งานจริง

2. ข้อกำหนดทางเทคนิค (Technical Requirements)

หัวข้อ	ข้อกำหนด
Backend Framework	Spring Boot 3.x+ (Java 17+)
Build Tool	Maven หรือ Gradle
Database	SQL เท่านั้น — PostgreSQL / MySQL / MariaDB
ORM	Spring Data JPA (Hibernate)
API	RESTful API + เอกสาร API (Swagger/OpenAPI)

Frontend	Thymeleaf หรือ React/Vue/Angular (เลือกอย่างใดอย่างหนึ่ง)
Testing	JUnit 5 + Mockito (+ Spring Boot Test)หรืออื่นๆ
Version Control	Git + GitHub (Public หรือเชิญอาจารย์เป็น Collaborator)
Deployment	ต้อง Deploy ขึ้น Cloud/Server ได้จริง

3. โครงสร้างโปรเจกแบบ Layered Architecture (บังคับ)

ต้องแยก Layer ชัดเจน และ ห้ามข้าม Layer (เช่น Controller เรียก Repository ตรง ๆ ถือว่าผิด)

Presentation Layer (Controller / RestController / Thymeleaf View)

↓

Service Layer (Business Logic, Transaction)

↓

Repository Layer (Data Access — Spring Data JPA)

↓

Domain / Entity (Entity, Value Object, Enum)
+ DTO Layer (Request/Response DTO + Mapper)
+ Config / Exception / Util / Security

ตัวอย่าง Package Structure

com.example.project

- |— config/
- |— controller/
 - | |— api/ # RestController
 - | |— web/ # Thymeleaf Controller
- |— service/
 - | |— impl/
- |— repository/
- |— domain/
 - | |— entity/
 - | |— enums/
- |— dto/
 - | |— request/
 - | |— response/
- |— mapper/
- |— exception/
- |— common/

4. SOLID Principles (ห้ามละเมิดทุกข้อ)

หลักการ	สิ่งที่ต้องแสดงให้เห็นในโค้ด
S — Single Responsibility	แต่ละ Class มีหน้าที่เดียว ไม่รวม Business + Validation + Persistence ในคลาสเดียว
O — Open/Closed	เพิ่มฟีเจอร์ใหม่ด้วยการเพิ่มคลาส ไม่ใช่แก้ if-else เดิม (ใช้ Strategy/Polymorphism)
L — Liskov Substitution	Subclass ใช้แทน Superclass ได้โดยไม่พัง ตรรกะ ไม่ throw UnsupportedOperationException

I — Interface Segregation	แยก Interface ย่อยตามการใช้งาน ไม่มี Fat Interface
D — Dependency Inversion	Service ขึ้นกับ Interface ไม่ใช่ Concrete Class + ใช้ Constructor Injection เท่านั้น

⚠ ต้องส่ง doc/solid-analysis.md ระบุว่าแต่ละหลักการปรากฏที่ไฟล์ไหน บรรทัดไหน พร้อมเหตุผลสั้น ๆ

5. Design Patterns (Checklist)

ต้องใช้ให้ครบตามตารางด้านล่าง และ อธิบายเหตุผลที่เลือกใช้ (ห้ามยัด Pattern มั่ว ๆ เพื่อให้ครบ — จะถูกหักคะแนน)

5.1 Enterprise / Architectural Patterns (บังคับทุกกลุ่ม)

- Layered Architecture — โครงสร้างโปรเจค
- MVC — Controller / Model / View
- Repository Pattern — Spring Data JPA
- Service Layer Pattern — รวม Business Logic
- DTO Pattern + Mapper — แยก Entity ออกจาก API Contract
- Dependency Injection — Constructor Injection

5.2 GoF Patterns (เลือกใช้ 1 กลุ่ม — อย่างน้อยกลุ่มละ 3 แบบ)

กลุ่ม	Pattern ที่แนะนำ	ตัวอย่างการใช้งานจริง
Creational	Singleton, Factory Method, Builder, Abstract Factory	@Bean Singleton, Factory สร้าง Payment/Report, Builder สร้าง Response DTO
Structural	Adapter, Facade, Decorator, Proxy, Composite	Adapter เชื่อม External API, Facade รวม Service หลายตัว, Proxy สำหรับ Caching
Behavioral	Strategy, Observer, Template Method, Command, State, Chain of Responsibility	Strategy คำนวณส่วนลด, Observer ส่ง Notification (ApplicationEvent), State จัดการสถานะ Order

 ต้องส่ง doc/design-patterns.md เป็นตาราง: Pattern | ปัญหาที่แก้ | ไฟล์/คลาสที่ใช้ | Class Diagram ประกอบ

6. ข้อกำหนดฐานข้อมูล

- ต้องมี อย่างน้อย 6 ตาราง
- ต้องมีความสัมพันธ์ครบทั้ง One-to-One และ One-to-Many (Many-to-Many เป็นคะแนนพิเศษ)
- ต้องมี Foreign Key Constraint, Index ที่เหมาะสม และกำหนด Cascade / Fetch Type อย่างมีเหตุผล
- ต้องมี ER Diagram และ Data Dictionary ในโฟลเดอร์ doc/

- ต้องมี Migration Script (Flyway หรือ Liquibase) หรืออย่างน้อย schema.sql + data.sql

ตัวอย่างความสัมพันธ์

- One-to-One: User UserProfile
- One-to-Many: Customer → Order → OrderItem

7. REST API Requirements

- ครบ CRUD อย่างน้อย 2 Resource หลัก
- ใช้ HTTP Method และ Status Code ถูกต้อง (200 / 201 / 204 / 400 / 404 / 409 / 500)
- ตั้งชื่อ Endpoint แบบ Resource-based:
/api/v1/customers/{id}/orders
- มี Global Exception Handler (@RestControllerAdvice) + Error Response Format มาตรฐาน
- มี Validation (@Valid, Bean Validation)
- มี Pagination & Sorting อย่างน้อย 1 Endpoint
- มี Swagger UI เข้าถึงได้จริง (/swagger-ui.html)

8. Git Workflow & กฎการทำงาน

8.1 รูปแบบชื่อ Branch (บังคับ ห้ามผิดรูปแบบ)

ชื่อ_รหัสนักศึกษา_section

ตัวอย่าง: somchai_66123456_01

Branch ที่ตั้งชื่อผิดรูปแบบ = ไม่ตรวจ และไม่ได้อะไรในส่วนนั้น

8.2 โครงสร้าง Branch

Branch	หน้าที่
main	Production — Merge ได้เฉพาะ Version ที่ส่งมอบ
develop	Integration — รวมงานจากทุกคน
ชื่อ_รหัส_section	Branch ส่วนตัวของแต่ละคน

8.3 กฎเหล็ก

- แต่ละคน Commit และ Push ด้วยบัญชี GitHub ของตนเอง เท่านั้น
- ห้ามฝากเพื่อน Commit / Push โดยเด็ดขาด
- ตั้งค่า git config user.name และ user.email ให้ตรงกับบัญชี GitHub ของตนเองก่อนเริ่มงาน
- ทุกคนต้องมี Commit ที่มีความหมาย ไม่น้อยกว่า 15 commits กระจายตลอดช่วงเวลาทำโปรเจค (ห้าม Commit รวดเดียวก่อนส่ง)

- การรวมงานต้องผ่าน Pull Request และมี Reviewer อย่างน้อย 1 คน ในทีม
- ห้าม Outsource / จ้างทำ / ให้ผู้อื่นนอกกลุ่มเขียนโค้ดให้ ไม่ว่ากรณีใด ๆ — ตรวจพบ = ตกทั้งกลุ่ม

8.4 Commit Message Convention

<type>: <สิ่งที่ทำ>

ตัวอย่าง: feat: add customer registration API
fix: correct order total calculation
refactor: extract discount strategy
interface
test: add unit test for OrderService
docs: update user manual

8.5 การให้คะแนนรายบุคคล

จะตรวจจาก git log, Contributors Graph, Pull Request และ Code Review ของแต่ละ Branch

-คะแนนแต่ละคนไม่เท่ากัน ขึ้นกับปริมาณและคุณภาพงานใน Branch ของตนเอง

9. โครงสร้าง GitHub Repository (ต้องครบทุกโฟลเดอร์)

└─  code/ # Source code + Configuration

└─  test # การทดสอบทั้งหมด

└─  doc/ # เอกสารทั้งหมดและสไลด์

└─  img/ # ไฟล์มัลติมีเดีย |

9.1 Diagram ที่ต้องมีใน doc/diagrams/

- Use Case Diagram + Use Case Description

- Domain Model / Conceptual Class Diagram
- Class Diagram (พร้อมแสดงตำแหน่ง Design Pattern)
- Sequence Diagram (อย่างน้อย 3 Scenario หลัก)
- Activity Diagram
- ER Diagram / Database Schema
- Component Diagram & Deployment Diagram
- State Diagram (ถ้ามี Entity ที่มีสถานะ)

10. ข้อกำหนด README.md

ต้องประกอบด้วยหัวข้อต่อไปนี้เป็นอย่างน้อย

ชื่อโปรเจค

คำอธิบายระบบสั้น ๆ 3-5 บรรทัด

สมาชิกกลุ่ม

| ลำดับ | ชื่อ-นามสกุล | รหัสนักศึกษา | Section | Branch | หน้าที่รับผิดชอบ |

Tech Stack

System Architecture

Database Design (ER Diagram)

Installation & Setup

How to Run

API Documentation

How to Run Tests

Deployment URL

Project Structure

11. Deployment

- ต้อง Deploy ให้เข้าถึงได้จริงผ่าน URL สาธารณะ
- ตัวเลือกที่แนะนำ: Render, Railway, Fly.io, Azure App Service, AWS EC2/Elastic Beanstalk, Google Cloud Run หรือ VPS + Docker
- ฐานข้อมูลใช้ Cloud DB ได้ (Supabase / Neon / Aiven / Railway Postgres)
- ต้องมี Dockerfile และ docker-compose.yml
- คะแนนพิเศษ: ตั้ง CI/CD ด้วย GitHub Actions (Build → Test → Deploy อัตโนมัติ)

12. ข้อกำหนดกลุ่ม

- สมาชิกกลุ่ม ไม่เกิน 5 คน
- ทุกคนต้องมียานเขียนโค้ดจริง (ไม่ใช่ทำแต่เอกสาร)
- ระบุหน้าที่รับผิดชอบของแต่ละคนใน README ให้ชัดเจน
- ส่งรายชื่อกลุ่ม + ลิงก์ Repository ภายในที่กำหนด

13. การหักคะแนน

กรณี	ผลลัพธ์
ชื่อ Branch ผิดรูปแบบ	-5 คะแนนรายบุคคล
ฝากเพื่อน Commit/Push	0 คะแนนในงานส่วนนั้น

Outsource / จ้างทำ	ตกทั้งกลุ่ม
ลอกโค้ดกลุ่มอื่น	ตกทั้งสองกลุ่ม
อธิบายโค้ดตัว เองไม่ได้	0 คะแนนใน ส่วนนั้น

14. Checklist ก่อนส่ง

- โฟลเดอร์ครบ: code/, test/, doc/, img/
- README.md มีตารางสมาชิกครบทุกคนพร้อมชื่อ Branch
- ทุกคนมี Branch ชื่อถูกรูปแบบ + Commit ≥ 15 ครั้ง
- Merge เข้า main ผ่าน Pull Request เรียบร้อย
- Deploy URL เปิดใช้งานได้จริง ณ วันนำเสนอ
- Swagger UI เข้าถึงได้
- Test ทั้งหมดรันผ่าน + มี Test Report
- เอกสาร Diagram ครบทุกชนิด
- Slide นำเสนออยู่ใน doc/slide/